DAY 25 | HANDS-ON ACTIVITIES

LLM Orchestration and FDE Skills 3b: Problem Statements

Six technical problems and one systems thinking activity. Every cell runs offline; add a key to see real tool choices.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Write Tool Schemas

Level: Warm-up | **Concepts:** JSON schema, descriptions

Write schemas for get_order_status(order_id), calculate_refund(amount, days_late) and create_ticket(summary, priority). Include descriptions with example values.

Sample run / expected result

```
{"name": "calculate_refund", "parameters": {"properties": {"amount": {"type": "number"}, ...}
```

Hints

- Use enum for priority: low, medium, high.

Problem 2: Validate Arguments

Level: Easy | **Concepts:** Pydantic, allowlist

Write run_tool(name, args_json) that rejects unknown tools and invalid arguments (bad order id format, negative amount) with clear error messages returned to the model.

Sample run / expected result

```
run_tool("delete_db", "{}") -> error: unknown tool
run_tool("calculate_refund", '{"amount": -5}') -> error: amount must be > 0
```

Hints

- Return errors as text so the model can recover.

Problem 3: The Tool Loop

Level: Medium | **Concepts:** multi-step tool calling

Answer "ORD-1042 arrived 3 days late, what refund do I get?" using two tool calls in sequence. Print every step.

Sample run / expected result

```
call get_order_status(ORD-1042) -> delivered, amount 4800
call calculate_refund(4800, 3) -> 720.0
answer: refund of 720
```

Hints

- Cap the loop at 5 steps.

Problem 4: Human Confirmation

Level: Medium | **Concepts:** risky actions

Mark create_ticket and issue_refund as risky. Before running them, ask for confirmation (input() or an auto-approve flag for tests) and log the decision.

Sample run / expected result

```
Model wants issue_refund(ORD-1042, 720). Approve? [y/N]: n
logged: rejected by user
```

Hints

- Default to no.

Problem 5: Support Graph

Level: Challenging | **Concepts:** LangGraph, conditional edges

Build a graph: classify -> (order | policy | other) -> answer -> check. If check fails, retry once with a rewritten question, then hand off to a human.

Sample run / expected result

```
classify: order -> lookup -> answer -> check: pass
classify: policy -> retrieve -> answer -> check: fail -> retry -> pass
```

Hints

- Keep attempts in state to stop loops.

Problem 6: Visualise the Graph

Level: Easy | **Concepts:** graph diagrams

Print the Mermaid diagram of your compiled graph and paste it into a Mermaid viewer or a Markdown file.

Sample run / expected result

```
graph TD; __start__ --> classify; classify -.-> order; ...
```

Hints

- app.get_graph().draw_mermaid()

Problem 7: Systems Map

Level: Group activity, 40 minutes | **Concepts:** causal loops, leverage points

A customer launched an AI chatbot and support tickets went up, not down. Draw a causal loop diagram with at least one reinforcing loop, one balancing loop and one delay. Choose a leverage point and test it with the notebook simulation.

Sample run / expected result

Reinforcing: wrong bot answers -> follow-up tickets -> longer queue -> more customers ask the bot -> more wrong answers
Balancing: queue grows -> overtime -> queue shrinks
Delay: knowledge base updates take 2 weeks
Leverage: daily review of failed bot answers feeding the knowledge base

Hints

- Arrows mean "influences"; mark each as same (+) or opposite (-) direction.

Nagare (流れ): flow